Story Point Estimation with BERT Word Embeddings — Final Report

PS-3: Evaluate BERT-based word embeddings for story point estimation when training on source projects and testing on a different target project under the assigned random-split protocol.

Group 12 — Challa Gouri Shankar (B230648CS), Konki Navadeep (B230386CS), E Srinith Reddy (B230929CS), Fahad Ahmed Mahdi (B230103CS), Anita (B230818CS)

GitHub: https://github.com/ettasrinith/story-point-estimation**

Report date: 10 November 2026

Executive summary

We predict the story point of a software issue from its title and description. For each of the 39 project pairs assigned in the guidelines, the model is trained on one project and tested on a different project it has never seen. We use frozen `bert-base-uncased` token embeddings and a small three-layer CNN, and report MAE and MdAE.

- **On the committed corpus (128-token window, story points `(0,40]`, 62,193 issues), the assigned**

PS-3 model scores 2.486 ± 1.303 test MAE against a 2.496 ± 1.450 median baseline, and improves mean per-pair MdAE from 1.872 to 1.681. It beats the baseline on 18 of 39 pairs.

- **Five-fold validation MAE is 1.809 ± 0.221, well below the 2.486 test MAE. The model is**

accurate inside its source project and degrades on an unseen project: transfer, not the architecture, is the limiting factor.

- **A separate extra experiment (not PS-3) trains each target on the other 38 projects and**

reaches 2.191 MAE. It changes three things at once, so this is an observation, not a claim that pooling alone caused the gain.

- **We repeated the assigned protocol with a 256-token window and the literature-standard**

`(0,100]` cap; all 39 pairs completed on a Kaggle T4 in 108.6 minutes. The longer window reduces truncation from 29.35% to 8.97% of issues, but the CNN does not beat its baseline on macro MAE: 2.966 ± 2.017 versus 2.897 ± 2.098, winning on 15 of 39 pairs (pair-level Wilcoxon p = 0.104). It does improve typical-case error — MdAE 1.792 vs 1.897 and issue-weighted MAE 3.961 vs 4.059.

- **So the measured benefit of 256 tokens is input fidelity, not accuracy. The cap is also not a**

cosmetic edit: it raises the median baseline from 2.496 to 2.897 MAE, because large story points inflate absolute errors. Because the corpus, cap and partitions all changed, old and new MAE are not a controlled window comparison.

1. Problem and scope

1.1 What PS-3 asks for

The assignment lists 12 problem statements. Ours is number 3:

Dimension	Our assignment
Text representation	**BERT word embeddings** (contextual token vectors)
Project setting	**Cross-project** — train on a source, test on a different target
Data splitting	**Random** 80:20, not chronological
Pairs	The **39** source → target pairs in guideline section 3.3
Metrics	**MAE** and **MdAE**, with five-fold validation reported separately from the held-out test

1.2 What is not in scope

SBERT sentence embeddings and combined BERT+SBERT representations belong to problem statements 5–12 and are handled separately. This repository never reports them. Guideline 3.4 asks groups to discuss "which representation achieves the lowest MAE/MdAE"; with only one assigned representation we can only compare BERT against the median baseline, and we say so rather than inventing rows for representations we did not run.

A CNN is allowed ("a three-layer CNN may be used", guideline 3.1), not mandated. We use one and document it fully.

2. Dataset

2.1 Extraction and cleaning

We used TAWOS (Tawosi et al., MSR 2022; DOI `10.5522/04/21308124`), a collection of Jira issues from 39 open-source projects. `01_extract_tawos.ipynb` installs a MySQL server inside a Kaggle notebook, loads the official dump, joins the Issue and Project tables, strips HTML from title + description, and exports one CSV per project. The result is 65,427 issues with a story point across 39 projects (project sizes range from 209 to 19,358 issues).

TAWOS also records whether an estimate was edited after it was made: 8,159 issues (12.5%) had their story point changed. We keep them, because the final value is the label we are asked to predict, but we report the count per project in `results/filter_impact.csv` because it bounds how noisy the target is.

2.2 Story-point filter — a documented design choice

The raw labels contain 2,416 zeros and values that are plainly data-entry errors; the largest is 26,710,114. Some filter is unavoidable, but the guidelines do not prescribe one. We therefore report both corpora and never compare their MAE as if the test sets were identical.

	Committed run	Redo
Filter	`(0,40]`	`(0,100]`
Issues kept	62,193	62,870 (+677)
Dropped above the cap	818	141
Median baseline MAE	2.496	2.897
Median baseline MdAE	1.872	1.897
Held-out issues across the 39 pairs	12,438	12,573

`(0,100]` is the de facto community ceiling: Deep-SE (Choetkiertikul et al., TSE 2019) drops zero, negative and "unrealistically large" (> 100) story points; the TAWOS README sample query uses `Story_Point BETWEEN 1 AND 100`; Tawosi et al. (ESEM 2022) remove SP < 1 and > 100. The original `(0,40]` cap was the outlier. Our `(0,100]` filter does **not** enforce the stricter Planning-Poker-card, estimate-never-updated or resolved-status criteria used in some replications; those need a corpus rebuild and are listed as a threat to external validity.

The cap mostly admits large story points. That is why MAE rises by 0.4 while MdAE barely moves — the median is robust to a long tail, the mean is not. Source: `results/256_sp100/audit/audit_corpus.json`, `baseline_comparison.csv`, `storypoint_histogram.csv`.

2.3 Matching the 39 pair names

Guideline 3.3 uses short labels such as `MESOS` and `Evergreen`. `tools/verify_project_mapping.py` resolves them in three machine-checked steps: **13 exact** matches, **24 normalised** short forms (`MESOS` → `Apache Mesos`), and **2 instructor-approved remaps**:

PDF label	TAWOS project
`Evergreen`	`The MongoDB Engineering`
`C++ driver (Appcelerator)`	`Aptana Studio`

Those are exactly the two TAWOS projects left over after exact + normalised matching, so the result is a verified bijection: 39 PDF labels → 39 distinct projects, each used once as a source and once as a target. See `results/project_name_mapping.csv` and `results/project_mapping_validation.json`.

3. Method

3.1 Text representation

`bert-base-uncased`, **frozen, inference only**. Each issue's title + description is tokenized and encoded once; the last hidden state gives one 768-dimensional vector per token. Arrays are stored per project as float16 and read with memory mapping during training, so BERT never runs twice.

	Committed	Redo
Maximum tokens	**128**	**256**
Array shape	`[n, 128, 768]`	`[n, 256, 768]`
Total size	**12.2 GB**	**24.7 GB**
Kaggle source	`tawos-bert-embed-v2`	`tawos-bert-256-a` + `-b`

The **padding mask** is applied before the first convolution and again after every convolution, so padded positions cannot leak into the pooled vector through a convolution's left context.

3.2 Predictor

A three-layer 1-D CNN over the token embeddings: `768 → 256 → 128 → 64` channels, kernel size 3, ReLU, dropout 0.2 on the first layer, masked global max pooling over real tokens, then an MLP head `64 → 32 → 1` with a **Softplus** output (a predicted story point can never be negative).

Loss is L1 (it matches MAE), optimiser Adam at `1e-3`, batch size 64, at most 25 epochs with early stopping patience 4 on validation MAE. All values are in `configs/ps3.yaml`.

3.3 Split and five-fold validation

For every project, issues are shuffled with `numpy.random.default_rng(42)` and split **80:20**. The 80% development portion is cut into **five folds**; each fold serves as validation once, and the best epoch is chosen on development data only. The final model trains for the **mean of the five best epochs** (8.8, range 2–16 in the committed run; 9.1, range 5–17 in the 256-token redo). The target project's held-out 20% is read **exactly once**, to score the finished model — never for training, epoch selection, early stopping or calibration. Validation and test metrics are reported in separate columns, as required by guideline 3.4.

The baseline is deliberately simple: predict the **source project's median story point** for every target test issue. A model that cannot beat this constant has learned nothing useful.

3.4 A separate extra experiment (not PS-3)

`06_pooled_training.ipynb` and `07_architecture_ablation.ipynb` ask a different question: for a given target, what if we train on the development pools of **the other 38 projects** instead of the single assigned source? This is the GPT2SP-style multi-source setup (Fu & Tantithamthavorn, TSE 2023). It also switches the target to `log1p(storypoint)`, averages three seeds (42/43/44), uses project-grouped cross-validation and adds a target-blind affine calibration.

The target's own development and test rows are still excluded from its own model, so there is no leakage — but **three things changed at once**. We therefore report these results separately and do **not** claim that pooling alone caused the improvement. `07` additionally swaps the CNN head for a one-layer Transformer encoder and a mean-pooled MLP; the three heads were statistically indistinguishable (Wilcoxon p = 0.92 for CNN vs Transformer, p = 0.31 for CNN vs MLP), which suggests the signal sits in the BERT embeddings rather than in the predictor.

4. Results

<!-- BEGIN GENERATED RESULTS --> _Auto-generated 2026-11-10 09:40 UTC by `redo/build_comparison.py`. Do not edit between the markers._

4.1 Headline tables

Committed run — 128 tokens, story points (0,40]

Configuration	Protocol	Test MAE	s.d.	Test MdAE	5-fold val MAE	Beats baseline	Test issues
Median baseline	**Constant baseline (no training)**	**2.496**	**1.450**	**1.872**	**—**	**—**	**12,438**
BERT + CNN (assigned PS-3)	**PS-3: 1 source → 1 target**	**2.486**	**1.303**	**1.681**	**1.809**	**18/39**	**12,438**
Pooled multi-source CNN	**Extension: 38 sources → 1 target**	**2.191**	**1.340**	**1.414**	**—**	**24/39**	**12,438**
Pooled Transformer head	**Extension: 38 sources → 1 target**	**2.242**	**1.260**	**1.511**	**—**	**23/39**	**12,438**
Pooled MLP head	**Extension: 38 sources → 1 target**	**2.216**	**1.285**	**1.463**	**—**	**23/39**	**12,438**

Redo — 256 tokens, story points (0,100]

Configuration	Protocol	Test MAE	s.d.	Test MdAE	5-fold val MAE	Beats baseline	Test issues
Median baseline	**Constant baseline (no training)**	**2.897**	**2.098**	**1.897**	**—**	**—**	**12,573**
BERT + CNN (redo PS-3)	**PS-3: 1 source → 1 target**	**2.966**	**2.017**	**1.792**	**1.991**	**15/39**	**12,573**

How to read these tables

- **Test MAE** is the mean of the 39 per-pair test MAEs; **s.d.** is the spread across those pairs. **Test MdAE** is the mean of the 39 per-pair median absolute errors.
- **PS-3: 1 source → 1 target** is the assigned protocol (guideline 3.3). **Extension: 38 sources → 1 target** is a separate extra experiment and must not be read as a PS-3 result or listed as a peer of the PS-3 row.
- **5-fold validation MAE** comes only from the source development pool and is reported separately from the single held-out target test, as required by guideline 3.4.
- The two tables use **different corpora** — 62,193 issues at (0,40] and 62,870 at (0,100]. Compare each model with **its own cohort's baseline**; an old-vs-new MAE difference is descriptive, not a controlled comparison of the token window.

4.2 Measured findings

- **Corpus.** (0,40] keeps 62,193 issues, (0,100] keeps 62,870 (+677); 141 values above 100 are still removed, including a 26,710,114 story-point data-entry error.
- **Window.** On the (0,100] corpus a 128-token window truncates 29.35% of issues and loses 41.66% of token content; 256 tokens reduce this to 8.97% and 26.20% (measured, not extrapolated).
- **Baseline moves with the corpus.** The same seed-42 median baseline scores 2.496 MAE on (0,40] but 2.897 on (0,100], while MdAE moves only 1.872 → 1.897. Never compare the two cohorts' MAE as if the test set were identical.
- **Redo PS-3 result (39/39 pairs).** At 256 tokens / (0,100], BERT + CNN scores macro test MAE 2.966 ± 2.017 against a 2.897 median baseline — above it by 0.069. It does **better on typical issues**: mean per-pair MdAE 1.792 vs 1.897, and issue-weighted MAE 3.961 vs 4.059. It wins on 15/39 pairs; five-fold validation MAE is 1.991.
- **No proven window gain.** The pair-level difference is not significant (Wilcoxon p = 0.104). Longer inputs preserve more text, but this experiment does **not** show a macro-MAE improvement; the corpus, cap and partitions all changed, so it is not a controlled 128-vs-256 A/B test.

Full detail: `results/comparison/COMPARISON.md`, **dashboard**, **master metrics CSV**, **all 39 pairs**. <!-- END GENERATED RESULTS -->

4.3 What the assigned PS-3 numbers mean

On the committed corpus the CNN's pair-average MAE (2.486) is essentially **tied** with the median baseline (2.496), but its typical error is clearly better: mean per-pair MdAE falls from **1.872 to 1.681** (−10.2%) and issue-weighted MAE from **3.323 to 3.185** (−4.2%). It wins on **18 of the 38 beatable pairs** (the 39th has a baseline MAE of zero — every held-out issue there has the same story point). So BERT carries real but modest signal under single-source transfer: the gain concentrates in the typical issue, not in the pairs with the largest story-point scales.

The worst pairs combine a source and target whose story-point scales differ sharply (a source with median 1–2 against a target with median 5). Text alone cannot recover the target's scale, so the model inherits a systematic offset. This is also why validation MAE (1.809) is far below test MAE (2.486): the **transfer gap**, not the network design, is the binding constraint.

On the **redo corpus** the same pattern appears, but more weakly. The CNN's macro test MAE (2.966) is **0.069 above** its 2.897 baseline and it wins on only 15 of 39 pairs; the pair-level Wilcoxon test gives p = 0.104, so neither direction is significant. Typical-case error again improves — MdAE 1.897 → 1.792 (−5.5%) and issue-weighted MAE 4.059 → 3.961 (−2.4%). The disagreement is explained by the long right tail admitted by the `(0,100]` cap: the single worst pair (`Blockchain Explorer → Moodle`) has test MAE 10.27 against a 10.26 baseline and dominates the macro mean. The median is robust to that tail; the mean is not.

The pooled extension reduces macro MAE to 2.191 and mean MdAE to 1.414, beating the baseline on 24 of 39 pairs. Its optional affine calibration turned out to be a no-op (2.191 → 2.195), which we report rather than present as a gain.

4.4 Validation versus held-out test

Quantity	Committed PS-3 (128 / (0,40])	Redo PS-3 (256 / (0,100])	Where it comes from
Five-fold validation MAE (best epoch)	**1.809 ± 0.221**	**1.991 ± 0.294**	**source development pool only**
Held-out target test MAE	**2.486 ± 1.303**	**2.966 ± 2.017**	**target 20%, read once**
Final epochs (mean of fold best)	**8.8 (2–16)**	**9.1 (5–17)**	**development folds only**
Held-out test issues	**12,438**	**12,573**	**39 target projects**

The gaps (1.809 → 2.486 and 1.991 → 2.966) are the cross-project generalisation error in each cohort. Guideline 3.2's requirement that model selection use development data only is what makes this comparison meaningful.

5. Discussion

Does the 256-token window help? It definitely improves *input fidelity*: on the `(0,100]` corpus it cuts the share of truncated issues from 29.35% to 8.97% and the lost token content from 41.66% to 26.20%. These are measurements, not extrapolations. **It does not, however, produce a measured macro-MAE gain.** The completed 39-pair CNN redo scores 2.966 against its own 2.897 baseline and wins on 15/39 pairs (Wilcoxon p = 0.104). Typical-case error does improve (MdAE 1.792 vs 1.897; issue-weighted MAE 3.961 vs 4.059). Two caveats prevent a stronger conclusion: the cap and the corpus partitions changed along with the window, so this is not a controlled 128-vs-256 A/B test, and a fixed BERT encoder with masked max pooling may simply discard the extra context rather than exploit it. We therefore report **better input fidelity, no demonstrated accuracy gain** — not an improvement we did not measure.

Does raising the cap to 100 help? It changes what is being evaluated, not just the amount of data. It admits 677 mostly large story points, which raises every method's MAE (including the constant baseline) while barely moving MdAE. `(0,100]` is the defensible, literature-standard choice; `(0,40]` was not. But an old-vs-new MAE comparison would be meaningless without also comparing each model to its own cohort's baseline.

Are the two cohorts paired? No. `rng.shuffle` walks the projects in sorted order using **one shared** `default_rng(42)` stream, and the `(0,100]` corpus changes the row count of 8 projects. Adding rows shifts

every later draw, so held-out rows move even where the held-out count does not. Within a cohort all models share partitions (so paired tests inside a cohort are valid); across cohorts they are not. Old-vs-new differences are descriptive only.

Which representation has the lowest error? Only BERT is assigned to us, so the only valid statement is BERT + CNN versus the median baseline: lower MdAE and issue-weighted MAE in **both** cohorts, with pair-average MAE essentially tied on `(0,40]` and slightly worse on `(0,100]`. SBERT and combined representations belong to other problem statements.

6. Threats to validity

- **Construct.** Story points are team-specific estimates, not objective effort. 12.5% of kept

issues had their estimate edited after the estimation timestamp, so part of the label noise is in the target itself.

- **Representation.** Even at 256 tokens, 8.97% of issues are truncated and 26.20% of token

content is lost, unevenly across projects. Long-description projects are penalised by the representation, not by their difficulty.

- **Internal.** Fixed seed and partitions for all methods; every training notebook verifies its

partitions against the recorded baseline before training. But the old/new corpora are not paired (see section 5), and the pooled extension confounds three changes at once.

- **Filter choice.** `(0,100]` uses `> 0` rather than `>= 1` and does not enforce Planning-Poker

scale membership, estimate-never-updated or resolved-status filters used in stricter replications.

- **External.** 39 pairs from one open-source Jira dataset; results may not generalise to

commercial trackers or other estimation scales.

- **Statistical.** One run per pair with a fixed seed. Fold variance is reported, but there are no

repeated global-seed runs; with 39 pairs and a 2.0 MAE spread in the redo cohort, individual pair differences are not significant (pair-level Wilcoxon p = 0.104 vs the baseline).

- **Window comparison.** Cap, token window, corpus size and partitions changed together between

cohorts, so the redo cannot isolate the effect of the 256-token window. A window-only ablation would need both views on one identical corpus and split; we did not run one and make no causal claim.

- **Completeness.** All 39 pairs of the 256-token CNN redo completed on a Kaggle T4 (108.6 min).

Every cell in every table is a measured value; no number was estimated or inserted while a run was unfinished (guideline 3.7).

7. Conclusion

Under the corrected protocol, frozen BERT word embeddings with a small CNN beat the median baseline on typical issues (MdAE 1.681 vs 1.872, −10.2%) while tying it on pair-average MAE (2.486 vs 2.496) across the 39 assigned cross-project pairs. The dominant obstacle is the transfer gap — validation MAE 1.809 versus test MAE 2.486 — not the choice of predictor head. A separate multi-source extension reaches 2.191 MAE but changes three variables at once, so it is reported as an observation rather than a controlled finding.

The 256-token / `(0,100]` redo preserves substantially more text (truncated issues 29.35% → 8.97%) and adopts the standard story-point ceiling. Its completed CNN run scores 2.966 ± 2.017 macro test MAE against a 2.897 ± 2.098 baseline and wins on 15 of 39 pairs, while improving MdAE to 1.792 and issue-weighted MAE to 3.961. **We therefore report a measured gain in input fidelity, not a gain in macro accuracy.** Because the cap and the partitions changed with the window, this is a faithful evaluation of the assigned PS-3 protocol on the corrected corpus, not a controlled test of window length. The transfer gap remains the binding constraint: 1.991 validation MAE versus 2.966 test MAE.

8. Reproducibility

All training ran on **Kaggle GPUs**; the local machine only builds kernels, downloads outputs and assembles reports. Kaggle account `madmaxop123`, dataset `ettasrinithreddy/tawos-extracted`.

Step	Notebook / kernel	Attach as input	Hardware	Recorded time
1	`01_extract_tawos.ipynb`	`TAWOS.sql` dataset	CPU + internet (apt)	not recorded
2	`02_baseline_median.ipynb`	output of 01	CPU	not recorded
3	`03_bert_embeddings.ipynb`	output of 01	T4 + internet (weights)	~2.2 min
4	`04_check_embeddings.ipynb`	output of 03	CPU	~0.3 min
5	`04_cnn_model.ipynb`	outputs of 01 + 03	T4	~2.5 min
6	`05_run_all_pairs.ipynb`	outputs of 01 + 03	T4	~65 min
7	`06_pooled_training.ipynb`	outputs of 01 + 03	T4	~2–2.5 h
8	`07_architecture_ablation.ipynb`	outputs of 01 + 03	T4	~3–4 h
9	`ps3-256-audit`	dataset	CPU	minutes
10	`ps3-256-main`	dataset + 256-token embedding kernels	T4	108.6 min (39/39 pairs)

Steps 5–8 also need `baseline_results.csv` from step 2 so that every model is compared against the baseline on the same partitions; each notebook asserts its own split matches the baseline before training. Kernels 01 and 02 no longer exist on the account — their outputs are published as the `tawos-extracted` dataset.

```
python tools/build_notebooks.py            # src/*.py -> *.ipynb
python tools/test_model_cpu.py             # CPU checks on masking / head / folds
python tools/verify_project_mapping.py     # re-check the 39 pair names
python redo/build_kernels.py               # rebuild the two redo kernels
kaggle kernels push -p redo/kernels/ps3-256-main
kaggle kernels output madmaxop123/ps3-256-main -p results/256_sp100/main
python redo/build_comparison.py            # regenerate all tables + dashboard
python redo/finalize_main.py               # one-shot: download + rebuild tables and PDF
```

Hyperparameters, seeds, split settings, model names and the rationale for the story-point cap are in `configs/ps3.yaml`. Per-pair CSVs and issue-level predictions live in `results/`; the comparative package is `results/comparison/`.

9. Guideline compliance

Guideline	Where it is satisfied
3.1 BERT representation, documented CNN, hyperparameters, seeds	**§3.1–3.2,** `configs/ps3.yaml`
3.2 80:20 split, five folds, development-only selection, isolated target test	**§3.3, §4.4**
3.3 the 39 assigned pairs, source/target documented	**§2.3,** `results/project_name_mapping.csv`
3.4 MAE + MdAE, validation separate from test, mean ± s.d.	**§4 (generated tables), §4.4**
3.5 README, source/notebooks, config files, results, comparative tables, plots	`README.md`, `configs/`, `results/comparison/`
3.6 GitHub link on the first page, PDF report, reproduction instructions	**header of this report,** `REPORT.pdf`**, §8**
3.7 actual findings only, no fabricated results	**all 7 rows measured, §4–§6**

References

- **Tawosi, V., Al-Helali, A., Sarro, F. — TAWOS dataset, MSR 2022. DOI** `10.5522/04/21308124`**.**
- **Choetkiertikul, R., Dam, H. K., Tran, T., Pham, T., Ghose, A., Menzies, T. — *A Deep Learning**

Model for Estimating Story Points*, IEEE TSE 2019 (Deep-SE).

- **Tawosi, V., Moussa, R., Sarro, F. — *Investigating the effectiveness of clustering for story**

point estimation*, ESEM 2022; and the Deep-SE replication using Porru et al.'s criteria.

- **Fu, M., Tantithamthavorn, T. —** ***GPT2SP: Cross-project story point estimation*****, IEEE TSE 2023.**
- **Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. — *BERT: Pre-training of Deep Bidirectional**

Transformers for Language Understanding*, NAACL 2019.